TP SonarQube *— Tests logiciels et assurance qualité*

1. Objectifs pédagogiques

- Configurer et compiler un projet Java avec Maven.
- Analyser un projet avec SonarQube et interpréter les résultats.
- Identifier des vulnérabilités, bugs potentiels et problèmes de maintenabilité.
- Corriger le code et justifier les choix techniques.
- Distinguer analyse statique, tests fonctionnels et mesure de couverture.

2. Contexte et consignes

L'application fournie contient des défauts de sécurité, de robustesse et de maintenabilité. Vous devez compiler le projet, effectuer une analyse SonarQube, documenter les résultats, corriger le code, puis comparer les analyses avant/après.

3. Prérequis

Java 17, Maven et une instance SonarQube accessible. Éditeur Java au choix.

4. Structure Maven

```
tp-sonarqube/
├ pom.xml
├ src/main/java/umbb/univ/tp/
      ├ App.java
      ├ DatabaseService.java
      ├ AuthService.java
      ├ OrderService.java
      ├ UserService.java
```

5. Fichier pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
   https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <groupId>umbb.univ</groupId>
   <artifactId>tp-sonarqube-qualite</artifactId>
   <version>1.0.0</version>
   <name>TP SonarQube - Qualité du code</name>
   <properties>
     <maven.compiler.release>17</maven.compiler.release>
     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
    <sonar.projectKey>tp-sonarqube-qualite</sonar.projectKey>
    <sonar.projectName>TP SonarQube - Qualité du code</sonar.projectName>
    <sonar.sourceEncoding>UTF-8</sonar.sourceEncoding>
  </properties>
  <build>
    <plugins><plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId><version>3.13.0</version>
    </plugin></plugins>
  </build>
</project>
```

Le code JDBC compile avec le JDK. Aucune base MySQL n'est nécessaire pour l'analyse statique. Pour exécuter réellement les requêtes, il faudra configurer une base de test et ajouter le pilote JDBC correspondant.

6. Code initial à analyser

DatabaseService.java

```java
package umbb.univ.tp;

import java.sql.*;

public class DatabaseService {
    public void getUser(String id) {
        try {
            Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/test", "root", "root123");
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(
                "SELECT * FROM users WHERE id=" + id
            );

            while(rs.next()) {
                System.out.println(rs.getString("name"));
            }

        } catch(Exception e) {
            e.printStackTrace();
        }
    }
}
```

AuthService.java

```java
package umbb.univ.tp;

import java.security.MessageDigest;

public class AuthService {

    private static final String ADMIN_PASSWORD = "admin123";

    public boolean login(String password) throws Exception {
        return hash(password).equals(hash(ADMIN_PASSWORD));
    }
    private String hash(String input) throws Exception {
        MessageDigest md = MessageDigest.getInstance("MD5");
        return new String(md.digest(input.getBytes()));
    }
}
```

OrderService.java

```java
package umbb.univ.tp;

public class OrderService {
    public double calculateDiscount(String type, double amount) {

        if(type.equals("VIP")) {
            if(amount > 1000) {
                return amount * 0.30;
            } else if(amount > 500) {
                return amount * 0.20;
            } else {
                return amount * 0.10;
            }
        } else if(type.equals("PREMIUM")) {
            if(amount > 1000) {
                return amount * 0.20;
            } else if(amount > 500) {
                return amount * 0.10;
            } else {
                return amount * 0.05;
            }
        } else {
            return 0;
        }
    }
}
```

UserService.java

```java
package umbb.univ.tp;

public class UserService {
    public void printRole(String role) {
        if(role.equals("ADMIN")) {
            System.out.println("Admin access");
        } else {
            System.out.println("User access");
        }
    }
    public void printRoleDuplicate(String role) {
        if(role.equals("ADMIN")) {
            System.out.println("Admin access");
        } else {
            System.out.println("User access");
        }
    }
}
```

App.java

```java
package umbb.univ.tp;

public class App {
    public static void main(String[] args) {

        DatabaseService db = new DatabaseService();
        db.getUser("1 OR 1=1");

        AuthService auth = new AuthService();
        try {
            auth.login("admin123");
        } catch(Exception e) {
            e.printStackTrace();
        }
    }
}
```

7. Travail demandé

Partie A — Analyse SonarQube initiale

1. Depuis la racine du projet, lancez l'analyse. Ne mettez jamais le jeton dans le code source, pom.xml ou un dépôt partagé.

Linux / macOS
export SONAR_TOKEN="votre_jeton"
mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
-Dsonar.host.url=http://localhost:9000 -Dsonar.token="$SONAR_TOKEN"

PowerShell
$env:SONAR_TOKEN="votre_jeton"
mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar `
-Dsonar.host.url=http://localhost:9000 -Dsonar.token="$env:SONAR_TOKEN"

2. Relever le statut du Quality Gate et les indicateurs principaux.
3. Pour chaque alerte pertinente, noter dans un tableau fichier, ligne, règle, sévérité, risque et correction proposée.
4. Consulter Bugs, Vulnerabilities, Security Hotspots et Code Smells ; joindre une capture ou un relevé.

Partie B — Diagnostic des défauts

Classe / zone	Risque à examiner	Conséquence possible
DatabaseService	Identifiants en dur ; concaténation SQL	Divulgation d'accès ; injection SQL
DatabaseService	Ressources JDBC non fermées	Fuite de ressources
DatabaseService	catch générique et printStackTrace	Gestion d'erreurs insuffisante
AuthService	Mot de passe en dur ; MD5	Secret récupérable ; hachage inadapté
AuthService	Conversion du condensat en String ; charset implicite	Représentation incorrecte/non portable
AuthService	Entrée password nulle	Risque de NullPointerException
OrderService	type nul ; règles métier et seuils à préciser	NPE ou résultats inattendus
OrderService	Conditions imbriquées et logique répétée	Maintenance plus difficile
UserService	Deux méthodes dupliquées	Corrections à répéter
App	Entrée « 1 OR 1=1 »	Démonstration d'une entrée hostile

Partie C — Amélioration du code

- Remplacer la concaténation SQL par `PreparedStatement` avec paramètre lié.
- Fermer `Connection`, `Statement` et `ResultSet` avec `try-with-resources`.

- Déplacer les secrets hors du code (variables d'environnement ou gestionnaire de secrets).
- Ne pas utiliser MD5 pour protéger les mots de passe : expliquer pourquoi Argon2id, bcrypt ou scrypt sont préférables. Ne pas créer sa propre cryptographie.
- Valider les entrées nulles/invalides et éviter les exceptions trop générales.
- Factoriser le code dupliqué dans `UserService`.
- Définir et documenter le comportement attendu pour les montants négatifs, nuls et les seuils de remise.
- Justifier les changements et conserver une séparation claire entre accès aux données, authentification et logique métier.